Trabajo de Criptografía pg 44 - 45

Ejercicio 1.6: Cifrado y Descifrado Simétrico (Cifrado César)

Este ejercicio utiliza el Cifrado César, un ejemplo clásico de criptografía simétrica por sustitución.

Parte 1: Cifrar el Mensaje "**SECRETO**" con **K=3**

- **Concepto de Clave (K)**

En criptografía, la **clave** (K) es una pieza de información secreta (un número, una palabra o una secuencia de bits) que, combinada con un algoritmo de cifrado, transforma el mensaje original (texto plano) en el mensaje cifrado (criptograma).

Significado de K=3 en el Cifrado César

En el contexto cifrado César, K=3 significa:

1. **Tipo de Clave:** Es la clave secreta compartida entre el emisor y el receptor (criptografía **simétrica**).
2. **Valor de Desplazamiento:** El número **3** indica cuántas posiciones se debe **desplazar** cada letra del mensaje original hacia adelante en el alfabeto para cifrar, y hacia atrás para descifrar.

Ejemplo Práctico

- **Cifrado (Desplazar hacia adelante):**

  Si la letra es **A** y usamos K=3, la letra cifrada será **D** (A -B - C - D).

- **Descifrado (Desplazar hacia atrás):**

  Si la letra cifrada es **D** y K=3, la letra original es **A** (D - C - B - A).

1. Tabla de asignación de números al alfabeto:

Letra	Número (P)	Letra	Número (P)	Letra	Número (P)
A	**1**	**J**	**10**	**S**	**19**
B	**2**	**K**	**11**	**T**	**20**
C	**3**	**L**	**12**	**U**	**21**
D	**4**	**M**	**13**	**V**	**22**
E	**5**	**N**	**14**	**W**	**23**
F	**6**	**O**	**15**	**X**	**24**
G	**7**	**P**	**16**	**Y**	**25**
H	**8**	**Q**	**17**	**Z**	**26**
I	**9**	**R**	**18**		

2. Conversión "SECRETO" a números:

Letra	**S**	**E**	**C**	**R**	**E**	**T**	**O**
Posición (P)	19	5	3	18	5	20	16

3. Aplicación la Fórmula de Cifrado:

Fórmula de Cifrado César (desplazamiento):

$$C = (P + K) \bmod N$$

Donde *N* es el tamaño del alfabeto (usaremos N=26). Si el resultado es 0, lo consideramos 26 (Z).

Letra original (P)	**19+3**	**5+3**	**3+3**	**18+3**	**5+3**	**20+3**	**16+3**
Resultado	22	8	6	21	8	23	19
Letra cifrada (C)	**V**	**H**	**F**	**U**	**H**	**W**	**S**

Respuesta a) ¿Cuál es el mensaje cifrado resultante?

- El mensaje cifrado es: **VHFUHWS**

Parte 2: Descifrar y Verificar el Mensaje con **K=3**

1. Aplicar la Fórmula de Descifrado

La fórmula para descifrar es:

$$P = (C - K) \bmod 26$$

Letra cifrada (C)	**V (22)**	**H (8)**	**F (6)**	**U (21)**	**H (8)**	**W (23)**	**S (19)**
C - 3	22-3	8-3	6-3	21-3	8-3	23-3	19-3
Resultado	19	5	3	18	5	20	16
Letra original (P)	**S**	**E**	**C**	**R**	**E**	**T**	**O**

Respuesta b) Descifra el mensaje cifrado y verifica si recuperas el mensaje original.

- El mensaje descifrado es **SECRETO**. Se recupera el mensaje original, verificando el proceso de cifrado y descifrado siendo estos correctos.

Ejercicio 1.7: Comparación entre Criptografía Simétrica y Asimétrica

Conceptos Clave:

- Simétrica (Clave Secreta Única): Usa la misma clave para cifrar y descifrar.
- Asimétrica (Par de Claves): Usa dos claves diferentes (una pública para cifrar y una privada para descifrar).

Característica	**Criptografía Simétrica**	**Criptografía Asimétrica**
Uso de claves	Se usa una sola clave (secreta) para cifrar y descifrar.	Se usa un par de claves (pública para cifrar, privada para descifrar).
Velocidad del cifrado/descifrado	Rápida (algoritmos más sencillos).	Lenta (algoritmos matemáticamente más complejos).
Seguridad frente a ataques	Alta, pero depende de la seguridad de la clave (si se filtra, la comunicación está comprometida).	Muy alta, protege la identidad y el no repudio (quien firma es quien dice ser).
Ejemplo de algoritmo	AES, DES, Twofish, Cifrado César.	RSA, ECC, ElGamal.

Ejercicio 1.8: Aplicaciones Prácticas de Criptografía

Este ejercicio evalúa cuándo es más apropiado usar la criptografía simétrica, asimétrica o una combinación.

Explicación:

- Simétrica: Se usa para confidencialidad y velocidad (cifrar muchos datos rápidamente).
- Asimétrica: Se usa para intercambio seguro de claves, firmas digitales, y autenticación (certificar la identidad).
- Combinación: La mayoría de sistemas modernos usan un híbrido (Asimétrica para intercambiar la clave secreta y Simétrica para cifrar los datos).

a) Una empresa quiere asegurar la transmisión de datos en tiempo real entre sus servidores, garantizando rapidez y seguridad.

- **Respuesta: Criptografía Simétrica (o un sistema Híbrido)**
    - Justificación: La rapidez es clave para la transmisión en tiempo real. La criptografía simétrica (como **AES**) es miles de veces más rápida que la asimétrica y es ideal para cifrar grandes volúmenes de datos de manera eficiente, garantizando la confidencialidad y la integridad. Un sistema híbrido usaría Asimétrica solo para el primer intercambio de la clave simétrica.

(AES) transforma bloques de datos de 128 bits (16 bytes) mediante una serie de sustituciones, permutaciones y transformaciones matemáticas a lo largo de varias "rondas", dependiendo del tamaño de la clave. El resultado es un criptograma que no tiene sentido sin la clave secreta correcta.

b) Un usuario quiere firmar digitalmente un contrato electrónico para asegurar que solo él pudo haber creado la firma.

- **Respuesta: Criptografía Asimétrica**
    - Justificación: La firma digital asegura el no repudio (no se puede negar haber firmado) y la autenticidad. Para esto, el usuario usa su clave privada (sólo él la tiene) para cifrar un ***hash*** del documento, y cualquier persona puede verificarlo usando su clave pública correspondiente.

c) Una aplicación de mensajería quiere asegurar que los mensajes enviados entre usuarios no puedan ser leídos por terceros y además verificar la identidad de los usuarios.

- **Respuesta: Una Combinación (Sistema Híbrido: Simétrico + Asimétrico)**
    - Justificación:
    - Simétrica (e.g., AES): Se usa para cifrar y descifrar el contenido de los mensajes (confidencialidad) debido a su alta velocidad.

- Asimétrica (e.g., RSA/ECC): Se usa para verificar la identidad de los usuarios y para intercambiar de forma segura la clave simétrica que se usará para la conversación.

Ejercicio 1.9: Resolviendo un Problema de Seguridad

Este ejercicio aborda un problema de seguridad común en la criptografía simétrica: el uso de claves débiles o cortas.

a) ¿Qué problemas de seguridad puede causar el uso de claves cortas en criptografía simétrica?

Problema Principal: Vulnerabilidad al Ataque de Fuerza Bruta

- **Explicación:** Una clave corta tiene un **espacio de claves** (el número total de claves posibles) muy pequeño.
  - Por ejemplo, una clave de 8 bits solo tiene $2^8 = 256$ combinaciones. Una computadora moderna podría probarlas todas en una fracción de segundo.
  - El atacante simplemente prueba **todas las combinaciones posibles de claves** hasta que encuentra la correcta que descifra el mensaje. Cuanto más larga sea la clave (e.g., 128 bits o 256 bits), el número de combinaciones es tan astronómico que el ataque de fuerza bruta se vuelve inviable.

b) Proporciona dos soluciones que podrías implementar para mejorar la seguridad sin cambiar completamente el sistema de cifrado.

Las soluciones deben mejorar la seguridad sin reemplazar completamente el algoritmo de cifrado simétrico (como pasar de AES a RSA, lo cual sería un cambio total).

1. Aumentar la Longitud de la Clave Secreta

- **Implementación:** Si el sistema actual usa una clave de 56 bits (como el antiguo DES), **migrar a una clave de 128 o 256 bits** (como AES) es el cambio más efectivo.
- **Impacto:** Esto incrementa el espacio de claves de forma exponencial, haciendo que un ataque de fuerza bruta sea **computacionalmente imposible** con la tecnología actual.

2. Usar Salt (Sal) e Iteraciones en la Derivación de Claves (Key Stretching)

- **Implementación:** Si la clave secreta se genera a partir de una contraseña de usuario:
    - **Salt:** Añadir una cadena de datos aleatoria y única (el "salt") a la contraseña antes de aplicar la función de *hashing* para crear la clave de cifrado.
    - **Iteraciones:** Repetir el proceso de *hashing* (iterar) cientos o miles de veces para ralentizar intencionalmente el proceso.
- **Impacto:** Esto dificulta enormemente los ataques que usan **tablas precalculadas** (como *Rainbow Tables*) y los ataques de **fuerza bruta en GPU**, ya que cada intento de adivinar la contraseña se vuelve miles de veces más lento.

Trabajo de Criptografía pg 90 - 91

Ejercicio 1.16. Comprensión de Conceptos Básicos

¿Qué es una Función Hash?

Una **Función Hash** es un **algoritmo matemático** que toma cualquier dato (entrada) y produce una salida de **tamaño fijo** llamada **resumen criptográfico** o **hash**. Es una **huella digital** única de esa información. No importa si la entrada es un documento pequeño o una película grande, el hash resultante siempre tendrá la misma longitud. Una característica crucial es que, para la misma entrada, siempre se obtiene el mismo hash.

Importancia Clave en Criptografía

La función hash es esencial para la seguridad digital debido a dos propiedades: **integridad** e **irreversibilidad**.

1. **Verificación de Integridad:** Se usa para asegurar que un archivo **no ha sido alterado**. Si cambias un solo *bit* en el archivo original, el hash resultante será **completamente diferente**. Si el hash original y el nuevo no coinciden, se confirma que la información no es íntegra (fue modificada).
2. **Almacenamiento Seguro de Contraseñas:** Las bases de datos guardan el **hash** de la contraseña, no la contraseña real. Esto es posible porque la función hash es **irreversible** (o de "sentido único"): es imposible obtener la contraseña original a partir del hash. Así, si la base de datos es robada, las contraseñas reales permanecen protegidas.

Ejercicio 1.17. Aplicación Práctica de Funciones Hash

Enunciado del Ejercicio

Considera que tienes un archivo de texto llamado **"datos.txt"** que contiene la frase **"Seguridad Informática 101"**. Utiliza un algoritmo hash como **SHA-256** o **MD5** para generar el resumen criptográfico de este archivo. ¿Cuál es el resultado?

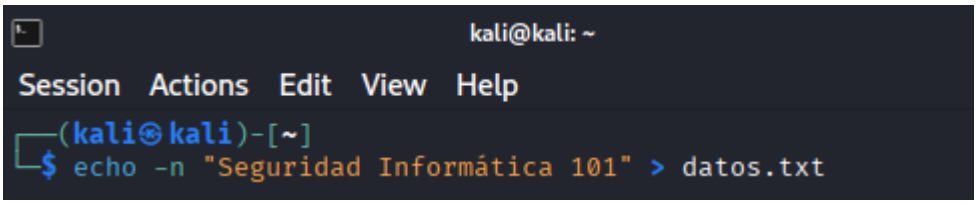
Instrucciones:

1. Usa una herramienta de línea de comandos para generar el hash.
2. Muestra los pasos seguidos para obtener el hash.
3. Compara los resultados obtenidos utilizando SHA-256 y MD5.

- **Pasos para Ejecutar en Kali Linux**

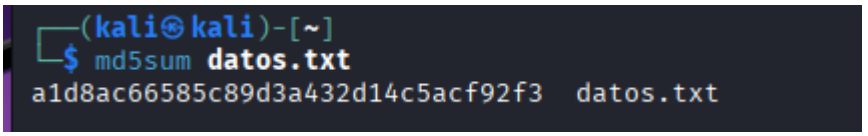
Paso 1: Crear el Archivo de Entrada

echo -n "Seguridad Informática 101" > datos.txt

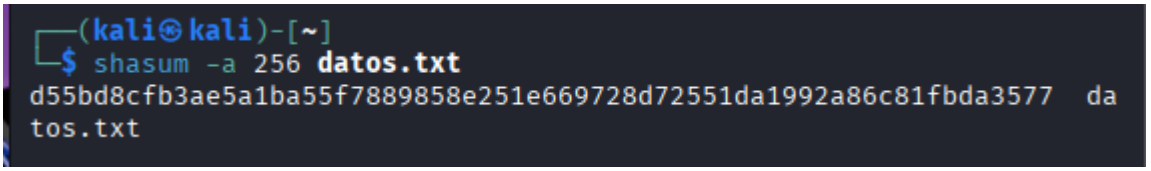


Paso 2: Generar el Hash MD5

md5sum datos.txt



Paso 3: Generar el Hash SHA-256



Paso 4: Comparación de Resultados

Algoritmo	Hash Generado	Longitud (Bits)
MD5	a1d8ac66585c89d3a432d14c5acf92f3	128
SHA-256	d55bd8cfb3ae5a1ba55f7889858e251e669728d72551da1992a86c81fbda3577	256

Conclusión y Comparación

La diferencia entre los dos conjuntos de hashes es una demostración práctica del Efecto Avalancha:

- MD5 y SHA-256 son funciones deterministas: Un cambio minúsculo en la entrada (un salto de línea) produce un hash final completamente diferente.
- Comparación: El hash SHA-256 (**d55bd8cfb3ae5a…**) es dos veces más largo que el hash MD5 (**a1d8ac665…**), lo que le confiere una seguridad mucho mayor y una resistencia superior a colisiones. Por ello, SHA-256 es el algoritmo preferido actualmente para la verificación de integridad.

Ejercicio 1.18. Evaluación de la Seguridad de Algoritmos Hash.

¿Por qué se desaconseja el uso de MD5 en aplicaciones críticas de seguridad?

Primeramente entramos en contexto:

- ### Conceptos Clave de Resistencia

Antes de comparar, definamos los dos tipos de ataques de resistencia:

Ataque	**Objetivo**	**Resistencia (Propiedad)**
Ataque de Colisión	Encontrar dos entradas **diferentes** (M_1 y M_2) que produzcan el **mismo hash** ($H(M_1) = H(M_2)$).	Resistencia a Colisión
Ataque de Preimagen	Dado un hash (H), encontrar la **entrada original** (M) que lo produjo, de forma que $H(M) = H$.	Resistencia a Preimagen (Irreversibilidad)

- ### Comparación de Fortalezas y Debilidades

Característica	**SHA-256 (Secure Hash Algorithm 256)**	**MD5 (Message Digest Algorithm 5)**
Longitud del Hash	256 bits (64 caracteres)	128 bits (32 caracteres)

Resistencia a Colisión	**Muy Alta** (Considerado criptográficamente seguro; es computacionalmente inviable encontrar colisiones).	**Baja/Nula** (Criptográficamente Roto; se han encontrado métodos rápidos para generar colisiones).
Resistencia a Preimagen	**Muy Alta** (Irreversible; la única forma práctica es mediante ataque de fuerza bruta).	**Alta** (Aún se considera difícil de revertir; sin embargo, las colisiones debilitan su uso en general).
Velocidad	**Moderada** (Más lento que MD5, pero más seguro).	**Muy Rápida**

En conclusión y teniendo en cuenta el anterior contexto:

MD5 se desaconseja totalmente para verificar la integridad de datos críticos (como firmware, certificados digitales, o para asegurar que un archivo no ha sido manipulado) debido a su falla total en la resistencia a colisiones.

- **El Problema**: La investigación ha demostrado que es factible (relativamente rápido y con hardware común) para un atacante generar dos archivos diferentes que resulten en el mismo hash MD5.
- El Riesgo (**Suplantación**): Un atacante podría crear un archivo malicioso ($\mathbf{M}_{\text{Malo}}$) y un archivo inofensivo ($\mathbf{M}_{\text{Bueno}}$) que comparten el mismo hash. Si el sistema de seguridad solo se verifica el hash MD5, el atacante puede hacer que la víctima descargue el archivo malicioso, y el sistema seguirá confirmando que el hash es correcto (porque coincide con el hash del archivo bueno).
- Conclusión: La integridad (la promesa de que los datos no han sido modificados) se rompe, haciendo que MD5 sea inutilizable para fines de seguridad crítica.

Por otro lado, SHA-256 pertenece a la familia SHA-2, que es el estándar actual por su robusta resistencia a ambos tipos de ataques.

Ejercicio 1.19. Caso de Estudio Integrador de Datos

Enunciado del Ejercicio

> Imagina que trabajas como **administrador de sistemas** en una empresa que debe garantizar la **integridad de los archivos descargados** por los usuarios.
>
> ¿Qué algoritmo hash recomendarías utilizar para verificar la integridad de los archivos? Justifica tu respuesta teniendo en cuenta aspectos como la **velocidad**, la **seguridad** y la **popularidad** del algoritmo.

Como administrador de sistemas encargado de la **integridad de archivos descargados**, se recomienda el uso del algoritmo **SHA-256** (Secure Hash Algorithm 256).

Justificación de la Elección

La elección de SHA-256 se justifica priorizando la **seguridad** y la **confianza** sobre cualquier pequeña ganancia de velocidad, garantizando la integridad de los datos de la empresa:

1. **Seguridad (Resistencia Criptográfica):** SHA-256 ofrece una **muy alta resistencia a colisiones** y a ataques de preimagen (irreversibilidad). Esto significa que es prácticamente imposible que un atacante cree un archivo malicioso que produzca la misma "huella digital" que un archivo legítimo. Esto es fundamental, ya que el objetivo principal es asegurar que los archivos **no han sido manipulados**.
2. **Popularidad y Estándar:** SHA-256 forma parte de la familia SHA-2, que es el **estándar actual** a nivel global. Está respaldado por gobiernos, navegadores y la mayoría de los protocolos de seguridad. Utilizarlo asegura **compatibilidad** y facilita la integración con otras herramientas de seguridad y verificación.
3. **Velocidad vs. Riesgo:** Aunque existen algoritmos más rápidos, la velocidad de SHA-256 es **eficiente** en el hardware moderno. La pequeña diferencia de tiempo que se gana con otros algoritmos no justifica el **riesgo inaceptable** de utilizar un algoritmo con vulnerabilidades conocidas en una aplicación de seguridad crítica.

Conclusión: SHA-256 es la opción más robusta y responsable para garantizar que cada archivo descargado sea **íntegro** y que provenga de la fuente esperada.

Ejercicio 1.20. Creación de una Función Hash Personalizada.

- **Diseño Conceptual: Función Hash Simple de Dígitos (HSD)**

Diseñaremos una función simple que toma un número entero (N) y lo transforma en otro valor de tamaño fijo.

Nombre: **F**unción Hash **S**imple de **D**ígitos (HSD)

Entrada (N): Cualquier número entero positivo (ejemplo: 987654)

Salida (H): Un valor de 3 dígitos (000 a 999).

Algoritmo Conceptual Paso a Paso:

1. **Cuadrado:** Se eleva el número de entrada al cuadrado: S = N^2
   - *Ejemplo:* 987654^2 = 975,460,904,016
2. **Suma y Recorte:** Sumamos todos los dígitos del resultado S.
   - *Ejemplo:* 9+7+5+4+6+0+9+0+4+0+1+6 = 51
3. **Compresión (Hashing):** Tomamos el resultado de la suma (51) y restamos el número 1. Luego, calcula el **Módulo 1000** (es decir, el resto de dividir entre 1000). Esto garantiza que la salida siempre tenga 3 dígitos.
   - *Ejemplo:* (51 - 1) mod 1000 = 50 mod 1000 = **050**

Paso Conceptual	Operación Matemática	Resultado Intermedio
1. Entrada (N)	Número de entrada	987654
2. Cuadrado (S)	$S = N^2$	$987654^2 = 975,460,904,016$
3. Suma y Recorte	Sumar todos los dígitos de S	$9+7+5+4+6+0+9+0+4+0+1+6 = 51$
4. Compresión (Hashing)	$(Suma - 1) \pmod{1000}$	$(51-1) \pmod{1000} = 50 \pmod{1000}$
5. Resultado Final (H)	Formato de 3 dígitos	050

Función Hash Simple de Dígitos (HSD) para la entrada 987654 es **050**

(Fácilmente reversible, muchas colisiones)

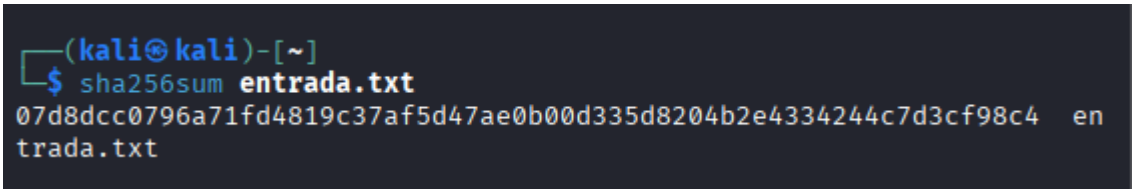
SHA-256 de Kali

Puesta en práctica:

- Paso 1: Generar la Entrada (N)



- Paso 2: Generar el Hash Criptográfico (H)



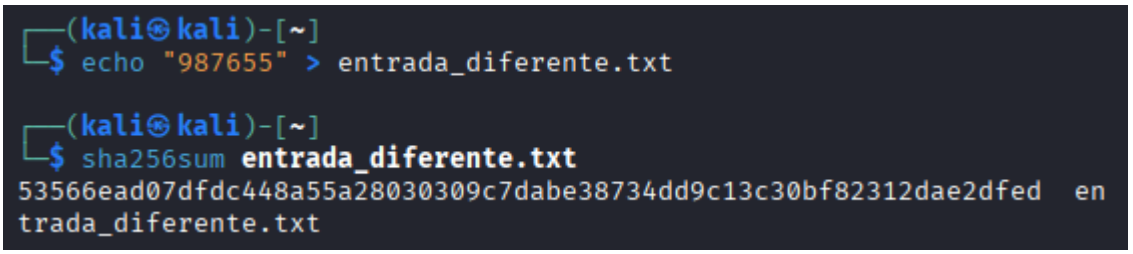
1. Demostración de Irreversibilidad

Una función hash como SHA-256 es una **función unidireccional** o de "calle de sentido único" .

1. **Transformación Destructiva:** El proceso de hashing mezcla y reduce la entrada (987654) aplicando transformaciones matemáticas tan complejas que **destruyen la información original** de una manera intencional.
2. **Imposible de Revertir:** Es trivial ir del número al hash, pero es **computacionalmente imposible** hacer el camino inverso (ir del hash al número). No existe un proceso matemático que pueda "deshacer" esa mezcla.
3. **La Única Opción:** La única manera de encontrar la entrada es adivinar y probar billones de combinaciones (fuerza bruta), lo cual tomaría un tiempo inviablemente largo (miles de millones de años).

Es decir que obtener el hash es fácil; obtener el número original a partir del hash es tan difícil que se considera **imposible** para cualquier computadora actual. Por eso se usa para proteger contraseñas.

2. Demostración de Resistencia a Colisiones



Comparación de Hashes

Entrada (N)	Hash SHA-256 (H)
Original (987654)	07d8dcc0796a71fd4819c37af5d47ae0b00d335d8204b2e4334244c7d3cf98c4
Nueva (987655)	53566ead07dfdc448a55a28030309c7dabe38734dd9c13c30bf82312dae2df

SHA-256 de Kali: Resultado `07d8dcc0796a71fd...` (Imposible de revertir, sin colisiones conocidas).

Resultado (Resistencia a Colisiones)

A pesar de que las entradas (987654 y 987655) son casi idénticas, **sus hashes son completamente diferentes** y no comparten ninguna coincidencia visible al principio.

Esto demuestra la propiedad de **Resistencia a Colisiones** (y el **Efecto Avalancha**):

- **Resistencia a Colisiones:** Es increíblemente difícil, o mejor dicho, **computacionalmente imposible**, encontrar dos entradas distintas que generen el mismo hash.
- **Efecto Avalancha:** Un cambio mínimo en la entrada provoca un cambio masivo e impredecible en la salida. Esto asegura que nadie pueda "adivinar" cómo modificar un archivo para obtener un hash predecible.

- ## Propiedades que debe tener una buena función Hash

Para ser considerada una **buena función hash criptográfica**, nuestro diseño debe aspirar a cumplir con tres propiedades clave, que son las que le dan valor para la seguridad digital:

Propiedad	**Descripción**
Uniformidad	Los resultados deben distribuirse equitativamente en todo el espacio de salida, **sin amontonarse** en unos pocos valores .
Irreversibilidad	Debe ser una **"calle de sentido único"**. Es fácil calcular el hash a partir de la entrada, pero computacionalmente es imposible encontrar la entrada original a partir del hash.
Resistencia a Colisiones	Debe ser **imposible** encontrar dos entradas diferentes que generen el mismo hash. Un cambio minúsculo en la entrada produce un hash completamente distinto (**Efecto Avalancha**).

Tabla de Contraste: HSD vs. SHA-256

Característica	HSD	SHA-256
Salida	Fijo (3 dígitos: 000-999)	Fijo (64 caracteres hexadecimales)
Uniformidad	**Mala**. Los resultados se agrupan en valores bajos.	**Excelente**. Cubre todo el espacio de salida de 2^{256}.
Irreversibilidad	**Nula**. Es trivial adivinar la entrada por la baja complejidad.	**Fuerte**. Computacionalmente imposible de revertir.
Resistencia a Colisiones	**Nula**. Es muy fácil encontrar entradas que sumen lo mismo.	**Fuerte**. Colisiones desconocidas y extremadamente improbables.

Trabajo de Criptografía pg 94

Ejercicio 1.21. Debate sobre Firmas Electrónicas vs. Manuscritas

Ventajas y Desventajas de las Firmas Electrónicas

Aspecto	Ventajas de la Firma Electrónica	Desventajas de la Firma Electrónica
Seguridad y Prueba	✅ **Mayor Nivel de Prueba:** Una firma digital avanzada ofrece **integridad** (se usa un hash para garantizar que el documento no cambió) e **identidad** (usa certificados para confirmar quién firmó). Es más difícil de falsificar que una firma a mano.	❌ **Dependencia de la Tecnología:** Requiere infraestructura (certificados, software, claves criptográficas). Si la tecnología falla o el certificado caduca, la firma puede invalidarse.
Eficiencia y Costo	✅ **Velocidad y Comodidad:** Permite firmar documentos al instante, sin importar la ubicación geográfica (lo que ahorra mucho tiempo y dinero en impresión y envío).	❌ **Curva de Aprendizaje:** Los usuarios menos técnicos pueden tener dificultades para entender cómo obtener, usar y proteger su certificado digital.
Integridad	✅ **Garantía de Integridad:** La firma digital **bloquea** el documento. Si se altera un solo carácter después de la firma, esta se invalida automáticamente.	❌ **Riesgos de Clave Privada:** Si la clave privada de un usuario es robada o comprometida, un atacante podría firmar documentos en su nombre.

¿Pueden las Firmas Electrónicas Reemplazar Completamente a las Manuscritas?

No, aún no pueden reemplazar completamente a las firmas manuscritas en ***todos*** los contextos legales, pero sí en la gran mayoría.

- **Ejemplos y Consideraciones Legales**

1. Contextos donde la firma electrónica domina:

En la mayoría de las situaciones cotidianas y comerciales, la firma electrónica (especialmente la **Firma Digital Avanzada** o **Cualificada**, que usa certificados) ya es el estándar y es legalmente superior:

- **Contratos Comerciales (B2B):** Es el método más rápido y seguro. Por ejemplo, una empresa de *software* firma un contrato de servicio con otra empresa a través de una plataforma digital. La ley (como la ley **eIDAS** en Europa o la ley **ESIGN** en EE. UU.) le da el mismo valor legal que una firma manuscrita.
- **Declaraciones de Impuestos:** El gobierno exige una forma de autenticación digital fuerte porque garantiza la identidad del declarante y la inalterabilidad de los datos enviados.

2. Contextos donde la firma manuscrita aún es relevante (las excepciones):
Existen áreas específicas donde las regulaciones o la tradición legal (y la necesidad de certeza física) aún exigen una firma manuscrita o, al menos, requieren una firma electrónica de la más alta categoría:

- **Testamentos y Herencias:** En muchos países, la ley requiere que el testamento sea firmado a mano por el testador y atestiguado por testigos **en presencia física** para evitar fraudes o presiones indebidas. La formalidad del papel es una barrera contra la manipulación digital.
- **Documentos Familiares Específicos:** Aunque esto cambia constantemente, ciertas actas matrimoniales o documentos de adopción pueden requerir la presencia física y la firma húmeda (manuscrita) por razones de derecho tradicional o para confirmar la voluntad en un acto solemne.
- **Documentos ante Notario (Actos de Fe Pública):** Aunque los notarios están adoptando la firma electrónica, el acto de firmar una escritura de propiedad o una hipoteca aún suele requerir la **firma física** o una **Firma Electrónica Cualificada** (el tipo de firma más seguro y con el máximo valor legal, equivalente a la firma manuscrita), y a menudo se hace en presencia del notario.

Conclusión Final:

Si bien las **firmas electrónicas son superiores** en términos de seguridad (*integridad criptográfica*) y eficiencia, no pueden reemplazar el componente **tradicional y presencial** que aún exigen ciertas leyes para proteger los derechos más sensibles de las personas (como la voluntad final o el estado civil). El futuro es electrónico, pero la transición total será lenta y estará marcada por las leyes de cada país.

Trabajo de Criptografía pg 104

Ejercicio 1.22. Concepto de Firma Digital

> Define el concepto de firma digital y explica cómo se diferencia de una firma manuscrita tradicional. ¿Qué elementos garantiza una firma digital en un documento electrónico?

¿Qué es una Firma Digital?

Una **firma digital** es una técnica criptográfica que se utiliza para garantizar la autenticidad y la integridad de un documento electrónico. No es una imagen de una firma a mano, sino un **código matemático** único que se genera usando la **clave privada** del firmante y un **hash** del documento.

Diferencia con la Firma Manuscrita

Característica	**Firma Manuscrita**	**Firma Digital**
Garantía Principal	Autenticación del *acto* (prueba de que la persona estaba allí y lo aprobó).	**Integridad** del *documento* y **Autenticación** del firmante.
Vinculación	Está vinculada a la **persona** (rasgos caligráficos).	Está vinculada al **contenido** (el hash) y a la **clave privada** del firmante.
Falsificación	Puede ser falsificada por un experto en caligrafía.	Es **criptográficamente imposible** de falsificar sin la clave privada del firmante.
Detección de Cambios	No detecta si se cambió el texto después de firmar.	Si se cambia **un solo carácter** del documento, la firma **se rompe**.

Elementos que Garantiza una Firma Digital

Una firma digital fuerte garantiza tres elementos esenciales de seguridad:

1. **Autenticidad:** Confirma **quién** firmó el documento. Se verifica que la clave privada utilizada para firmar pertenece realmente al firmante declarado.
2. **Integridad:** Garantiza que el documento **no ha sido alterado** desde el momento en que se firmó. Si hay un cambio, el hash de verificación fallará.
3. **No Repudio:** El firmante **no puede negar** haber firmado el documento. Puesto que solo él posee la clave privada, la ley lo responsabiliza por la acción.

Ejercicio 1.23. Tipos de Firma Digital

Clasifica los siguientes escenarios según el tipo de firma digital más adecuado (simple, avanzada, cualificada):

- Un usuario firma un **contrato de arrendamiento en línea**, y se necesita que esta firma tenga la **misma validez legal** que una firma manuscrita.
- Un cliente aprueba una **transacción bancaria** a través de una aplicación móvil.
- Un empleado firma electrónicamente su **asistencia diaria** en el trabajo.

Para clasificar, usaremos la normativa europea **eIDAS** (que es un estándar internacional), donde la seguridad y validez legal crecen de simple a cualificada:

Tipo de Firma	**Descripción para Principiantes**	**Valor Legal**
Simple	Cualquier dato electrónico (Ej: escribir tu nombre al final de un email o un *checkbox* "Acepto").	Bajo (Fácil de impugnar).
Avanzada	Vinculada al firmante y puede detectar cambios. Requiere un **certificado digital** asociado a la identidad.	Alto (Tiene la misma validez legal que la

		manuscrita en muchos casos).
Cualificada	Es una **Firma Avanzada** creada con un dispositivo de creación de firma certificado (como un *hardware token*) y respaldada por un certificado cualificado.	Máximo (Legalmente equivalente a la firma manuscrita en toda la UE).

Clasificación de Escenarios

Escenario	Clasificación Recomendada	Justificación
Contrato de arrendamiento en línea que necesita la misma validez legal que una firma manuscrita.	**Avanzada o Cualificada**	Necesita el máximo nivel de **no repudio** y **autenticidad** para un documento con implicaciones legales serias (contratos, *pólizas*).
Un cliente aprueba una **transacción bancaria** a través de una aplicación móvil.	**Avanzada**	Requiere una identificación robusta (huella dactilar, clave OTP o *token* de seguridad), que detecte el **fraude**.
Un empleado firma electrónicamente su **asistencia diaria** en el trabajo.	**Simple**	La importancia legal es baja. Simplemente se requiere verificar la **identidad** del usuario al momento de registrar el evento, lo cual se logra con un nombre de usuario y contraseña junto a una marca de tiempo.

Ejercicio 1.24. Uso y Validación de Firmas Digitales

> Describe el proceso de validación de una firma digital. ¿Qué pasos se deben seguir para verificar que una firma digital es válida? Además, ¿qué rol juegan las autoridades de certificación (CA) en este proceso?

1. Proceso de Validación (Verificación)

La validación de una firma digital es un proceso de dos pasos que utiliza la clave pública del firmante para garantizar la integridad y la autenticidad del documento.

1. **Paso 1: Verificar la Integridad del Documento:**
    - El verificador (el receptor) calcula el **hash** del documento **recibido**.
    - Luego, utiliza la **clave pública** del firmante para **descifrar** el hash que está *incrustado* en la firma.
    - Si el **hash calculado** es **igual** al **hash descifrado**, se confirma que el documento **no ha sido alterado**.
2. **Paso 2: Verificar la Autenticidad del Firmante (y el Certificado):**
    - El verificador revisa el **certificado digital** del firmante (que contiene la clave pública).
    - Se verifica que el certificado **no haya expirado** y que **no haya sido revocado** (cancelado).

Pasos de Verificación:

Fase 1: Creación de las Claves y el Archivo de Entrada

1. Generar una clave privada (la "llave secreta" del firmante)

La clave se guarda en 'private.key'

```
┌──(kali㉿kali)-[~]
└─$ openssl genrsa -out private.key 2048
```

2. Extraer la clave pública correspondiente (la "llave de candado")

La clave se guarda en 'public.pem'

```
┌──(kali㉿kali)-[~]
└─$ openssl rsa -in private.key -outform PEM -pubout -out public.pem

writing RSA key
```

3. Crear el documento original a firmar

```
┌──(kali㉿kali)-[~]
└─$ echo "El contrato de 1000 euros queda aceptado y sellado." > con
trato.txt
```

Fase 2: Firmar el Documento (Acción del Firmante)

4. Firmar el documento (crear el hash cifrado)

```
┌──(kali㉿kali)-[~]
└─$ echo "── Documento firmado. Ahora verificaremos su validez. ──
"
── Documento firmado. Ahora verificaremos su validez. ──
```

Fase 3: Verificación de la Integridad (Acción del Verificador)

5. Verificar la firma usando la clave pública

```
┌──(kali㉿kali)-[~]
└─$ openssl dgst -sha256 -verify public.pem -signature contrato.sig
contrato.txt
Verified OK
```

PRUEBA 2: Verificación Fallida (Documento Alterado)

Simularemos un ataque de alteración del documento para demostrar la Resistencia a Colisión (Efecto Avalancha).

6. Alterar el documento original (cambiamos "euros" por "dólares")

```
┌──(kali㉿kali)-[~]
└─$ echo "El contrato de 1000 dolares queda aceptado y sellado." > c
ontrato.txt
```

7. Intentar verificar la firma con el documento alterado

```
┌──(kali㉿kali)-[~]
└─$ openssl dgst -sha256 -verify public.pem -signature contrato.sig
contrato.txt
Verification failure
4037A8B72F7F0000:error:02000068:rsa routines:ossl_rsa_verify:bad sig
nature:../crypto/rsa/rsa_sign.c:442:
4037A8B72F7F0000:error:1C880004:Provider routines:rsa_verify_directl
y:RSA lib:../providers/implementations/signature/rsa_sig.c:1041:
```

- Un documento firmado exitosamente es validado (`Verified OK`).
- El más mínimo cambio en el documento rompe la integridad y anula la firma (`Verification Failure`).

2. El Rol de las Autoridades de Certificación (CA)

Las **Autoridades de Certificación (CA)** juegan un rol de **tercero de confianza** y son fundamentales para la autenticidad.

- **Emisión de Certificados:** La CA es una entidad de confianza (como el gobierno o una empresa acreditada) que verifica la **identidad real** de la persona u organización. Una vez confirmada, la CA emite el **Certificado Digital** (que incluye la clave pública) y lo firma con su propia clave privada.
- **Garantía de autenticidad:** La firma de la CA en el certificado garantiza al mundo que la clave pública efectivamente pertenece a la persona que dice ser. Esencialmente, la CA dice: "Yo, la Autoridad de Confianza, confirmó que esta persona es quien dice ser."
- **Listas de Revocación:** Las CA mantienen una lista pública de certificados que han sido revocados (por robo de clave, cese de actividad, etc.), asegurando que los certificados inválidos no puedan usarse.



Ejercicio 1.25. Ventajas y Desventajas de las Firmas Digitales

Discute las principales ventajas y desventajas de las firmas digitales en comparación con las firmas manuscritas tradicionales. ¿En qué casos podría ser preferible una firma digital?

1. Ventajas Clave (¿Por qué la preferimos?)

- **Seguridad Superior:** La firma digital ofrece una garantía de **integridad inigualable**. Es la única que puede demostrar criptográficamente que un documento no ha sido modificado.
- **Eficiencia:** Permite firmar contratos o documentos con personas al otro lado del mundo **instantáneamente**, eliminando los costos y el tiempo del correo físico.
- **Autenticación Fuerte:** Vincula la acción a una **clave privada única**, haciendo que el **no repudio** sea legalmente mucho más fácil de probar que la simple caligrafía.

2. Desventajas clave (limitaciones)

- **Dependencia Tecnológica:** Requiere **certificados**, *software* y acceso a internet. Un fallo en la infraestructura hace imposible firmar o verificar.
- **Costo Inicial y Mantenimiento:** Obtener y mantener certificados digitales cualificados puede tener un costo, a diferencia de la firma a mano.
- **Problemas de Almacenamiento:** El usuario es responsable de proteger su **clave privada**; si esta se pierde o es robada, el atacante puede firmar en su nombre.

3. Casos donde la firma digital es preferible

La firma digital es **siempre preferible** en contextos que requieren **velocidad, distancia y máxima garantía de integridad** del contenido.

Caso de Uso	Justificación de la Preferencia
Transferencias Bancarias/Financieras	Se requiere una verificación inmediata y no se puede permitir que la orden de pago sea alterada ni por un segundo.
Documentación Legal de Larga Duración	Un documento legal que debe durar 30 años (como un título de propiedad) tiene mayor garantía de que no será alterado si se sella digitalmente.
Licitaciones/Contratos a Distancia	Permite a empresas de diferentes países cerrar acuerdos al instante con plena validez legal.

Trabajo de Criptografía pg 114

Trabajo de Criptografía pg 114

Ejercicio 1.26: Diferencias entre Cifrado de Flujo y Cifrado de Bloque

Explica las principales diferencias entre el cifrado de flujo y el cifrado de bloque. Asegúrate de cubrir los siguientes aspectos:

1. Modo de operación
2. Ventajas y desventajas
3. Ejemplos de algoritmos que usan cada uno de estos métodos.

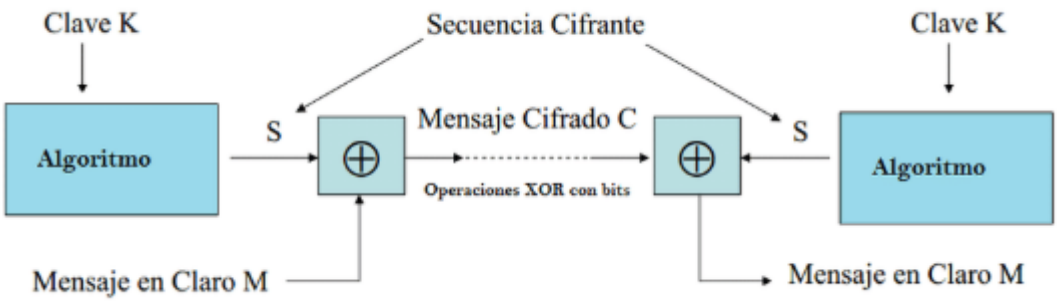
Aspecto	**Cifrado de Flujo**	**Cifrado de Bloque**
1. Modo de Operación	Cifra **bit** a **bit** o **byte** a **byte**.	Cifra datos en **bloques de tamaño fijo** (ej. 128 bits).
2. Ventaja Principal	Velocidad y Eficiencia (baja latencia).	Seguridad Estructural (mejor para datos fijos).
2. Desventaja Principal	Riesgo de Reutilización de Clave (catastrófico).	Requiere Relleno (Padding, aumenta la sobrecarga).
3. Ejemplos	ChaCha20, Salsa20	**AES** (Advanced Encryption Standard)

Ejercicio 1.27: Análisis de Casos de Uso (Video en Tiempo Real)

Se presenta un escenario en el cual necesitas asegurar una transmisión en tiempo real de video a través de Internet. ¿Qué tipo de cifrado (de flujo o de bloque) elegirías para proteger la transmisión y por qué?

- Se elegiría el **cifrado de Flujo**. La prioridad es la baja latencia. El cifrado de flujo procesa los datos de forma continua (*bit a bit*), lo cual es ideal para el *streaming* de video, evitando la espera de bloques completos que genera latencia.

https://www.researchgate.net/figure/Figura-1-Esquema-general-del-Cifrado-de-Flujo_fig2_333701648

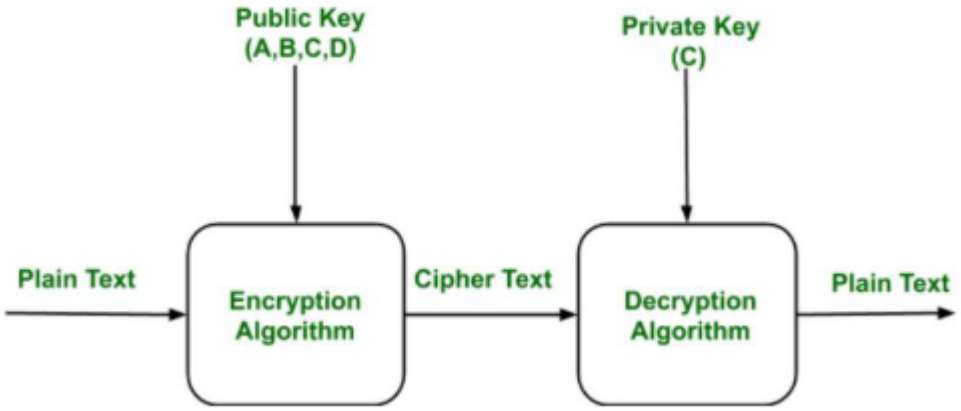


Ejercicio 1.28: Comparación de Protocolos de Intercambio de Claves

Compara los protocolos Diffie-Hellman y RSA en términos de:

1. Mecanismo de generación de claves
2. Uso de claves públicas y privadas
3. Seguridad y resistencia frente a ataques
4. Casos de uso más comunes.

Aspecto	**Diffie-Hellman (DH)**	**RSA**
Generación	Basado en el **Logaritmo Discreto** .	Basado en la **Factorización de Primos** (ej. N = p x q).
Uso de Claves	Solo para **acordar una clave** secreta (Intercambio).	Cifrado**, Descifrado y Firmas Digitales**.
Seguridad	Ofrece **Secreto Adelantado Perfecto (PFS)**.	La seguridad se basa en la dificultad de factorizar.
Casos de Uso	**Establecimiento de Sesiones Seguras** (en TLS/VPN).	**Cifrado de Datos/Certificados Digitales** (Firma de CA).



https://www.geeksforgeeks.org/computer-networks/difference-between-diffie-hellman-and-rsa/

Análisis del Diagrama:

1. **RSA (Lado Izquierdo):** El diagrama muestra un proceso directo de cifrado y descifrado de datos. La Clave Pública se usa para cifrar un mensaje, y la Clave Privada correspondiente se usa para descifrarlo. RSA puede proteger el contenido del mensaje.
2. **Diffie-Hellman (Lado Derecho):** El diagrama muestra un proceso de intercambio de claves. El objetivo NO es cifrar un mensaje completo, sino que dos partes (Alice y Bob) colaboren para generar y acordar una clave secreta simétrica compartida (**K**) a través de un canal inseguro.

Conclusión:

- **DH:** Su uso es la generación de la llave maestra. Solo se usa para intercambio de claves.
- **RSA:** Su uso es la protección del contenido. Se usa para cifrar datos o crear/verificar firmas.

Ejercicio 1.29: Aplicación práctica de ECDH

Supón que estás implementando un sistema de comunicación seguro para un dispositivo IoT con recursos limitados. Explica por qué elegirías el protocolo ECDH para el intercambio de claves y describe cómo funcionaría.

1. **¿Por Qué Elegir ECDH (Curva Elíptica Diffie-Hellman)?**

Elegiría ECDH sobre RSA o el Diffie-Hellman (DH) estándar por su Eficiencia Criptográfica Superior .

Ventaja Clave	**Impacto Directo en Dispositivos IoT**
Tamaño de Clave Reducido	ECDH logra el **mismo nivel de seguridad** que RSA, pero con claves significativamente más cortas (ej. 256 bits en ECDH es tan seguro como 3072 bits en RSA).
Bajo Consumo de Energía	Las operaciones de Curva Elíptica requieren **menos cálculos computacionales**, lo que se traduce en un **menor consumo de batería** y CPU.
Alta Velocidad	El intercambio de claves es **mucho más rápido** que con RSA o DH grandes, ideal para sesiones rápidas o intermitentes.

Funcionamiento de ECDH (El Intercambio "Liviano")

ECDH permite a dos partes acordar una clave simétrica secreta a través de un canal público e inseguro.

Paso	Parte A (Dispositivo IoT)	Parte B (Servidor)	Función en la Seguridad
1. Secreto Local	Elige un número privado/secreto d_A.	Elige un número privado/secreto d_B.	Base de la seguridad; este número nunca se comparte.
2. Clave Pública	Calcula el **Punto Público**: $P_A = d_A \times G$.	Calcula el **Punto Público**: $P_B = d_B \times G$.	G es un punto base de la curva conocida. Calcular el punto es fácil; revertirlo (encontrar d) es difícil.
3. Intercambio	Envía: P_A a B.	Envía: P_B a A.	Intercambian los puntos públicos.
4. Clave Compartida	Clave Compartida: $K = d_A \times P_B$.	Clave Compartida: $K = d_B \times P_A$.	Ambos llegan al mismo punto K. Este punto es la **Clave Maestra Secreta Simétrica**.

El resultado K (la Clave Compartida) se utiliza entonces para cifrar los datos de la sesión usando un algoritmo rápido como ChaCha20 o AES-128.

Prueba Práctica de ECDH (Intercambio de Claves)

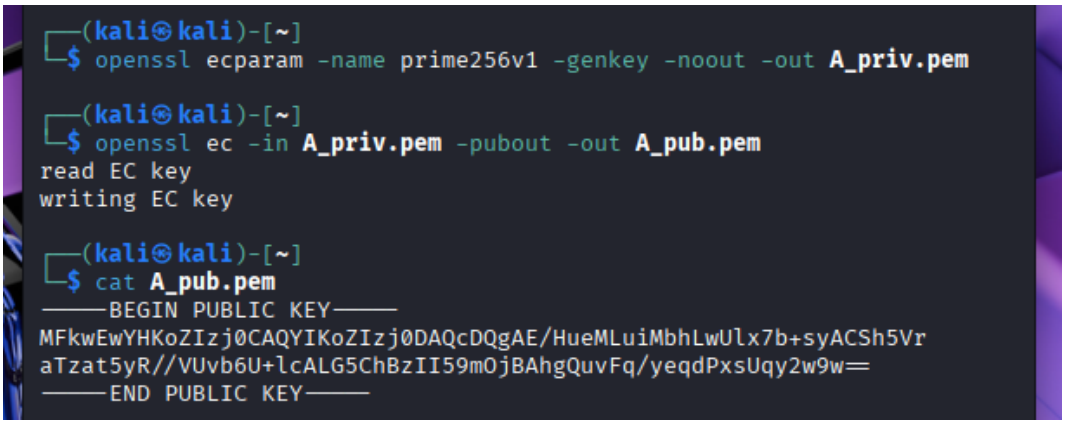
Contexto:

- IoT A (Cliente)
- Servidor B

FASE 1: Generación de Claves (En cada Máquina)

Paso	Terminal A (IoT)	Terminal B (Servidor)	Notas
1. Generar Privada	openssl ecparam -name prime256v1 -genkey -noout -out A_priv.pem	openssl ecparam -name prime256v1 -genkey -noout -out B_priv.pem	Crea el secreto privado (d_A y d_B).
2. Generar Pública	openssl ec -in A_priv.pem -pubout -out A_pub.pem	openssl ec -in B_priv.pem -pubout -out B_pub.pem	Extrae la clave pública (P_A y P_B).
3. Leer Pública	cat A_pub.pem	cat B_pub.pem	CRÍTICO: Muestra la clave pública para el intercambio manual.

Máquina A:



Máquina B:

```
┌──(kali㉿kali)-[~]
└─$ openssl ecparam -name prime256v1 -genkey -noout -out B_priv.pem

┌──(kali㉿kali)-[~]
└─$ openssl ec -in B_priv.pem -pubout -out B_pub.pem
read EC key
writing EC key

┌──(kali㉿kali)-[~]
└─$ nano A_pub.pem
```

```
┌──(kali㉿kali)-[~]
└─$ cat B_pub.pem
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEVfYrzwSYmirF3waHzE3CCZYXDoPd
OagbzSsDAMCNyiK3/TrOA5p0BuCMaMv8HSVDghwt8pmbnI2pLogT8BchMg==
-----END PUBLIC KEY-----
```

FASE 2: Intercambio Manual (La Clave del Éxito)

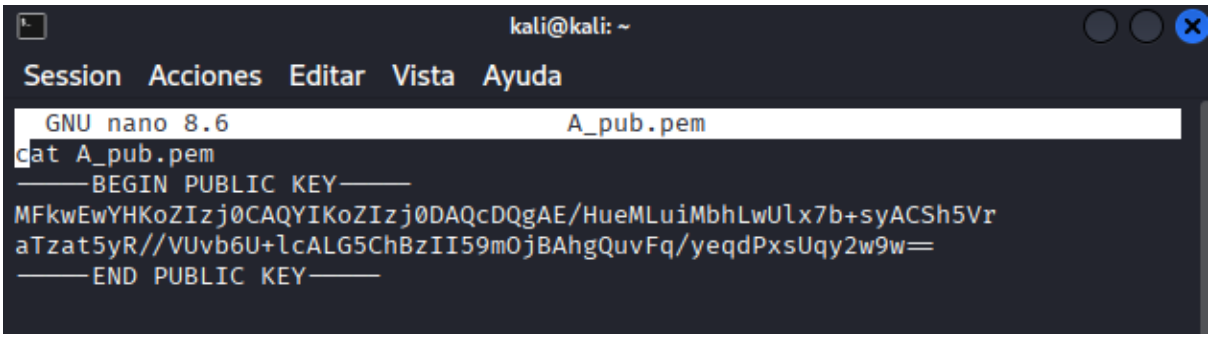
Copiar el texto completo de P_A (del Terminal A) y pegarlo en un nuevo archivo llamado (A o B)_pub.pem en el Terminal B (y viceversa para P_B).

Máquina A:

- nano B_pub.pem

```
kali@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                    B_pub.pem
cat B_pub.pem
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEVfYrzwSYmirF3waHzE3CCZYXDoPd
OagbzSsDAMCNyiK3/TrOA5p0BuCMaMv8HSVDghwt8pmbnI2pLogT8BchMg==
-----END PUBLIC KEY-----
```

Máquina B:



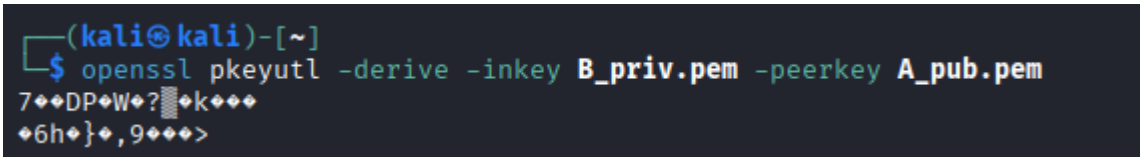
FASE 3: Cálculo del Secreto Compartido K (La Prueba)

Verificamos que ambos equipos llegaron a la misma clave secreta K usando la fórmula: **K = d_{propia} x P_{ajena}**

Terminal	Comando Ejecutado	Resultado (Secreto Compartido)
Terminal B (Servidor)	**openssl pkeyutl -derive -inkey B_priv.pem -peerkey A_pub.pem**	**7••DP•W?•k•••••6h•,•,9••• **
Terminal A (IoT)	**openssl pkeyutl -derive -inkey A_priv.pem -peerkey B_pub.pem**	**7••DP•W?•k•••••6h•,•,9••• **

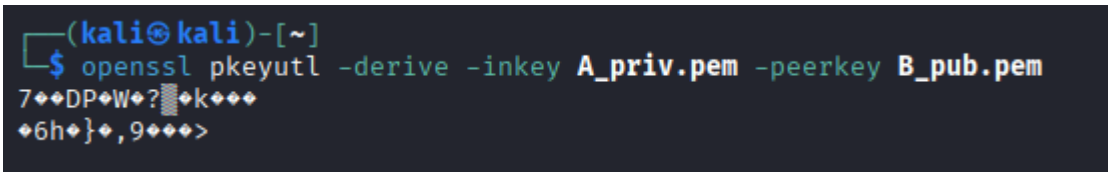
Máquina B:

- openssl pkeyutl -derive -inkey B_priv.pem -peerkey A_pub.pem



Máquina A:

- openssl pkeyutl -derive -inkey A_priv.pem -peerkey B_pub.pem



Los resultados binarios son **idénticos**. Esto demuestra exitosamente que:

1. El protocolo **ECDH** funciona: Dos partes (A y B) acordaron un secreto compartido a través de un canal inseguro.
2. El uso de claves de **Curva Elíptica (prime256v1)** es ideal para IoT debido a su eficiencia.
3. Se ha realizado la prueba en dos máquinas virtuales de Kali Linux.

Ejercicio 1.30: Implementación y seguridad en protocolos de intercambio de claves

Identifica los riesgos de seguridad del protocolo **Diffie-Hellman sin autenticación** y propón una solución para mitigarlos.

1. El Riesgo Fundamental: Man-in-the-Middle (MitM)

El principal problema de Diffie-Hellman (DH) en su versión pura (sin certificados) es la **falta de autenticación**.

Riesgo	**Descripción**	**Consecuencia**
Ataque Man-in-the-Middle	El atacante (M) se interpone en la comunicación. Cuando A envía su clave pública (P_A), M la intercepta y le envía a B su propia clave pública (P_M), y viceversa.	M establece dos secretos compartidos: K_{AM} (con A) y K_{BM} (con B). M puede descifrar, leer y volver a cifrar **todos los mensajes** sin que A o B lo detecten.

Falsificación de Identidad	A y B creen que están hablando entre sí, cuando en realidad están hablando con el atacante.	La comunicación se asegura, pero no se asegura la **identidad** de las partes.

2. Solución Propuesta: Infraestructura de Clave Pública (PKI)

La forma de asegurar DH/ECDH es complementarlo con un mecanismo que pruebe la identidad.

Solución	Mecanismo de Mitigación
Certificados Digitales	La clave pública de A y B se incluye en un **Certificado Digital** que está firmado por una **Autoridad de Certificación (CA)** de confianza (ej. Verisign).
Rol de la CA	Cuando B recibe la clave pública de A, B verifica la firma del certificado con la clave pública de la CA. Si la firma es válida, B sabe que la clave pública le pertenece legítimamente a A.
Mitigación del MitM	El atacante (M) no puede obtener un certificado válido de una CA para su clave falsa. Por lo tanto, el receptor detecta la falsificación y **termina la conexión inmediatamente**, frustrando el ataque.

Aplicación Práctica: Esto es exactamente lo que hace **TLS/SSL** (el protocolo que usa HTTPS): utiliza DH o ECDH para el **intercambio eficiente de la clave de sesión**, pero utiliza certificados (generalmente RSA o ECDSA) para la **autenticación** inicial.